Web Music Player

Project Proposal & User Stories

HSLU · Web Programming Lab · 2026-09-01

1 Introduction

A web-based music player. Users upload MP3 files, organise them into playlists, and play them back on a screen styled as a record player — a vinyl disc that spins while a track plays and a tone-arm that drops on play and lifts on pause. The two self-defined CRUD resources are **Songs** and **Playlists**.

2 Tech stack

Concern	Choice
Frontend	Angular
Backend	Express.js (REST)
Database	PostgreSQL — song/playlist metadata
Audio storage	Server filesystem (mounted volume); DB holds the path
Tests	Jest (unit), Supertest + real PostgreSQL (integration), Playwright (E2E)
Packaging	Docker Compose — `docker compose up`

3 Committed stories

3.1 Songs

SNG-1 — Manage songs (CRUD) Must

As a user, *I want* to upload, browse, edit and delete songs, *so that* I control my own library. **Create** — upload `.mp3` only (MIME + extension), files over 20 MB rejected; ID3 tags (title, artist, album, duration, cover art) pre-fill the record, a missing title falls back to the filename; the file lands on the uploads volume with a metadata row in PostgreSQL. · **Read** — a sortable list shows title, artist, album, duration and date added (sortable by title, artist, date); cover thumbnails have fixed dimensions and lazy-load; empty state links to upload. · **Update** — title, artist and album are editable; title is required, duration is read-only; the audio file is never modified. · **Delete** — requires confirmation; removes the DB row, the file on disk and the song from every playlist; if the song is playing, playback advances or stops.

SNG-2 — Play a song in the record-player view Must

As a user, *I want* to play a song in the record-player view, *so that* listening feels like a real turntable. Vinyl disc with the cover art as its label, plus a tone-arm; the disc spins only while playing and the tone-arm lifts on pause. · Always-visible controls: play/pause, previous, next, seek bar, current / total time. · `prefers-reduced-motion` disables the spin animation. · Audio is served with HTTP Range support so seeking works on mobile.

SNG-3 — Store MP3 files in S3-compatible object storage Could

As a operator, *I want* uploaded MP3 files to live in an S3-compatible bucket instead of the local volume, *so that* the app can run on more than one node and survive container restarts without a shared disk. The storage backend is chosen by configuration — local filesystem or an S3-compatible bucket (AWS S3, MinIO, …); the default stays local so `docker compose up` needs no cloud account. · Upload writes the object to the bucket; playback streams it back with HTTP Range support (presigned URL or proxied through the API); delete removes the object with no orphans left behind. · The S3 path is covered by integration tests running against a local MinIO container.

3.2 Playlists

PL-1 — Manage playlists (CRUD) Must

As a user, *I want* to create, edit and delete playlists and their songs, *so that* I can organise songs into sets. **Create** — name required, duplicate names allowed; the playlist appears in the overview immediately, empty. · **Read** — the overview lists each playlist with its track count; opening one shows its ordered songs; empty states for "no playlists" and "empty playlist". · **Update** — rename with the same name rules; add songs from the song list or playlist view (a song may sit in several playlists, appended to the end); remove songs (affects only this playlist, remaining order preserved). Changes are reflected in every view; removing the playing song advances playback. · **Delete** — requires confirmation; the songs stay in the library; playback stops if that playlist is playing.

PL-2 — Play a playlist with auto-advance Must

As a user, *I want* to play a playlist and have it advance automatically, *so that* I can listen without touching the device. Opens the record-player view on the first or selected track; the next track starts automatically when one ends. · Previous / next move within the playlist; playing a single library song behaves as a playlist of one.

PL-3 — Reorder songs in a playlist Should

As a user, *I want* to change the order of songs in a playlist, *so that* the playlist flows the way I want. Drag-and-drop on desktop, a touch control on mobile; the new order persists and drives auto-advance.

3.3 Playback

PB-1 — Seek within a track Must

As a user, *I want* to jump to any position in the current track, *so that* I can skip or replay a part. Click or drag the seek bar to set the position; the current-time display follows; works on mobile.

PB-2 — Repeat mode Should

As a user, *I want* to toggle repeat between off, all and one, *so that* I can loop a playlist or a track. Repeat-all wraps past the last track; repeat-one restarts the current track; the mode is shown and persists across reload.

PB-3 — Volume control Should

As a user, *I want* to set playback volume in the app, *so that* I don't need OS controls. A 0–100% slider plus a mute toggle that remembers the previous level; persists across reload.

PB-4 — Resume after reload Should

As a user, *I want* the app to remember what I was playing after a reload, *so that* a refresh doesn't lose my place. Restores the current playlist, track and position, paused.

3.4 Authentication

AUTH-1 — Register and sign in Should

As a visitor, *I want* to register with email and password and sign in, *so that* my library is private to me. Email validated for format and uniqueness; minimum password length; passwords stored only as bcrypt hashes. · A session cookie is set on sign-in and cleared on sign-out. No email verification or password reset (known limitation).

AUTH-2 — Per-user data Should

As a user, *I want* my songs and playlists to be visible only to me, *so that* other users can't see or change my data. Every song and playlist belongs to a user; all endpoints filter by the authenticated user; other users' resources return 403/404. · With auth disabled, one implicit local user owns everything.

3.5 Usability

NFR-1 — Feedback, empty states and accessibility basics Should

As a user, *I want* clear feedback and keyboard support throughout, *so that* the app is pleasant for everyone. Async actions show loading plus a success or error result; every list has a designed empty state. · Primary flows are keyboard-operable with visible focus; images have alt text; contrast meets WCAG AA.